TECHNICAL PROJECT REPORT

FastAPI Observability, Monitoring & Real-Time Alerting

Implementation guide, architecture, alert rules, failure testing, and maintenance

PROJECT DETAILS	REFERENCE
Repository	github.com/ghkrknkp/FAST-API
Walkthrough video	Watch project walkthrough on Loom
Local dashboard	http://localhost:8000/dashboard (after starting the application)
Documentation scope	FastAPI API metrics, health checks, dashboard, alert evaluation, Slack/email dispatch, and simulated incidents

Purpose. This document explains how to set up, operate, test, and maintain the monitoring project. Configuration names and routes reflect the supplied project description and should be checked against the repository before a production deployment.

Document Overview

This report is organized for a technical reviewer who needs to understand the system, run it locally, verify the key behaviors, and troubleshoot common issues.

Section	What it covers
1. Executive summary	Project goals and monitoring approach
2. Setup and access	Prerequisites, installation, environment configuration, local URLs
3. Architecture	Request instrumentation, system metrics, endpoints, notification channels
4. Dashboard	Status indicators, metrics, alerts, and test controls
5. Alert rules	Conditions, severity, and recommended response
6. Failure testing	Reproducible scenarios and expected telemetry
7. Troubleshooting	Common local setup and notification problems
8. Maintenance and security	Dependencies, logs, secrets, and SLO review
9. Verification checklist	Items to confirm before submission or handover
10. References	Repository, dashboard, API docs, metrics, and video walkthrough

1. Executive Summary

The project demonstrates observability for a FastAPI REST application using the RED method: request Rate, request Errors, and request Duration. It combines request-level instrumentation, a live dashboard, health information, threshold-based alert evaluation, and notification dispatch through Slack and email.

The intended outcome is to help a developer or operator notice abnormal API behavior, inspect the relevant telemetry, and follow a consistent response path. The project also includes simulated failure routes so that error, latency, authentication, and rate-limit behavior can be demonstrated without relying on a real production incident.

> Important scope note: the supplied description documents the intended design and expected results. A report alone does not prove that every feature is implemented or currently working. Confirm each route, metric name, alert transition, and notification against the checked-out code and a live test run before describing it as verified.

2. Setup and Access

2.1 Prerequisites

- Python 3.11 or later.
- Git installed and available from the terminal.
- cURL or another HTTP client for endpoint testing.
- Slack webhook and SMTP credentials only if notification delivery is being tested.

2.2 Install the project

```
git clone https://github.com/ghkrknkp/FAST-API.git
cd FAST-API

python -m venv venv

# Windows CMD
venv\Scripts\activate

# macOS / Linux
source venv/bin/activate

python -m pip install --upgrade pip
pip install -r requirements.txt

# Create a local environment file if the example exists
# Windows CMD:
copy .env.example .env
# macOS / Linux:
cp .env.example .env

uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

If the repository does not contain `.env.example`, do not run the copy command as written. Check the repository for its documented environment-variable template and create `.env` accordingly.

2.3 Local access points

Service	URL	Purpose
Monitoring dashboard	http://localhost:8000/dashboard	Dashboard view for API status, metrics, and alert cards
Swagger UI	http://localhost:8000/docs	Interactive API documentation and endpoint testing
ReDoc	http://localhost:8000/redoc	Alternative API documentation
Prometheus metrics	http://localhost:8000/metrics	Exposed metrics in Prometheus text format, if configured
Health check	http://localhost:8000/health	Application health probe

Use `127.0.0.1` or `localhost` for local access from the same machine. Binding Uvicorn to `0.0.0.0` makes it listen on network interfaces; do not expose a development server publicly without appropriate access controls.

3. Monitoring Architecture

3.1 Request lifecycle

```
Client
  |
  v
FastAPI ASGI application (Uvicorn :8000)
  |
  +--> ASGI instrumentation middleware
  |      +--> request counter (rate / status codes)
  |      +--> request duration histogram
  |      +--> in-flight request gauge
  |
  +--> API routes and health checks
  |
  +--> System collector (for example, psutil)
  |      +--> CPU utilization
  |      +--> process or system memory usage
  |
  +--> Exposed endpoints
         +--> /metrics
         +--> /api/alerts
         +--> /dashboard
                  |
                  +--> Slack notification dispatcher
                  +--> Email / SMTP dispatcher
```

3.2 Main components

Component	Responsibility
FastAPI + Uvicorn	Serves API routes and hosts the ASGI application.
ASGI metrics middleware	Observes requests and records counters, duration, and in-flight activity.
System collector	Collects host/process metrics such as CPU and memory, if enabled in the implementation.
/metrics endpoint	Publishes instrumentation in a Prometheus-compatible exposition format, if configured.
Alert evaluator	Compares measured values or event rates with configured thresholds.
/dashboard	Presents current status, metrics, alert states, and test actions.
Slack / SMTP dispatchers	Send notifications when dispatch is configured and credentials are valid.

3.3 RED method

- **Rate:** how many requests the service receives over time.
- **Errors:** the count or rate of failed requests, commonly grouped by HTTP status.
- **Duration:** how long requests take, usually measured with a histogram to support latency analysis.

Prometheus-style metric names and labels must be consistent with the actual code. Avoid high-cardinality labels such as raw user IDs, request IDs, or full URL paths containing arbitrary identifiers.

4. Dashboard Configuration

The dashboard at `/dashboard` is intended to provide a single view of service health and recent behavior.

Dashboard element	What it should communicate
Target status (UP / DOWN)	Whether the application target responds to its status check.
Health state	Whether the configured application health probe considers the service healthy.
Uptime	Elapsed time since the current process or service instance started.

Dashboard element	What it should communicate
Total requests	Cumulative request count represented by the instrumentation.
Active alerts	Number of alert rules currently in a firing state.
Traffic generator	Controls for exercising normal and simulated-failure routes.
Alert cards	Alert name, severity, condition, current state, and suggested remediation.
Notification test controls	Manual tests for Slack or email dispatch, if implemented and enabled.

Dashboard values depend on how the frontend refreshes data and how the backend stores metrics. Confirm the polling interval, error handling, and whether counters reset after a restart by inspecting the implementation.

5. Configured Alert Rules

The following rules reflect the supplied project specification. Threshold expressions and metric names should be reconciled with the actual alert engine before treating this table as a production configuration.

Alert	Severity	Condition / threshold	Suggested response
HTTP500Error	Critical	HTTP 500 response rate greater than 0 req/s	Inspect application logs and the failing route; reproduce using the error simulation endpoint.
SlowAPIResponse	Warning	Request latency exceeds 3.0 seconds	Check slow downstream I/O, database queries, and the timeout simulation route.
ServiceHealthFailure	Critical	Health metric equals 0	Inspect dependencies and readiness logic; use the health toggle only in a test environment.
AuthenticationFailureSpike	Warning	401 rate exceeds 0.1 req/s	Check client credentials and investigate unusual authentication failures.
RateLimitExceeded	Warning	429 rate exceeds 0.1 req/s	Review request volume and quota behavior; check that clients respect Retry-After.
APIDown	Critical	No response for more than 15 seconds	Check process status, logs, host resources, port binding, and restart history.

Operational considerations

- A condition such as “rate greater than zero” can fire on a single error; use a deliberate evaluation window and alert grouping to avoid noisy notifications.
- A process that is down cannot reliably send its own “API down” notification. Detecting full unavailability generally requires an external monitor or separate monitoring process.
- Thresholds are examples, not universal SLOs. Choose them based on traffic, latency objectives, and acceptable error budgets.

6. Failure Simulation and Verification

Run simulated failures only in a local or isolated test environment. After each request, inspect the response, the metrics endpoint, the dashboard, and the alert state.

Scenario	Example request	Expected behavior to verify
HTTP 500	GET /simulate-error	A controlled 500 response; the 500-response counter increases; the corresponding alert becomes firing if its rule is satisfied.
Slow response	GET /simulate-timeout?delay=3.5	The request takes about 3.5 seconds; duration observation records the latency; slow-response alert evaluates accordingly.

Scenario	Example request	Expected behavior to verify
Authentication failure	GET /protected without X-API-Key	A 401 response; authentication-failure telemetry increases if instrumented.
Rate limit	GET /simulate-rate-limit	A 429 response; rate-limit telemetry increases. Verify Retry-After only if the route actually sets it.

Example cURL commands

```
# General health check
curl -i http://localhost:8000/health

# Trigger a controlled server error
curl -i http://localhost:8000/simulate-error

# Trigger a slow response
curl -i "http://localhost:8000/simulate-timeout?delay=3.5"

# Trigger an authentication failure
curl -i http://localhost:8000/protected

# Trigger a rate-limit response
curl -i http://localhost:8000/simulate-rate-limit

# Inspect exposed metrics and alert state
curl -s http://localhost:8000/metrics
curl -s http://localhost:8000/api/alerts
```

Metric names mentioned in the original specification include `fastapi_requests_total`, `fastapi_request_duration_seconds`, and `fastapi_auth_failures_total`. Confirm the exact names, labels, and bucket behavior from the actual output of `/metrics`.

6.1 Suggested test record

Field	Record for each test
Test name and time	Name of scenario and timestamp
Request	Method, URL, and relevant headers/parameters
Observed response	HTTP status and response body summary
Metric change	Metric name and before/after value or sample
Alert state	Normal / pending / firing, with evaluation delay
Notification	Slack/email received, or error details if not received

7. Troubleshooting

Problem	Likely cause	Action
Port 8000 is already in use	Another process is listening on the port.	Windows CMD: `netstat -ano | findstr :8000`. Identify the PID, then stop only the process you intend to terminate: `taskkill /PID PID_NUMBER /F`.
Slack notification not received	Webhook missing, invalid, blocked, or rate-limited.	Check the webhook environment variable, dispatcher logs, network access, and the provider's response. Never paste the webhook into a public issue.
Email delivery fails	Incorrect SMTP settings, authentication, TLS, or account policy.	Verify host/port, TLS behavior, sender/recipient, and use an app password where the account provider requires it.
Dashboard does not update	Frontend polling error, API error, or mismatched response schema.	Open browser developer tools, inspect network requests, and compare the response with what the UI expects.
Uptime returns to zero	Application process restarted or reloaded.	Check Uvicorn logs and file changes. Development reloads reset process-local uptime.
Alert never fires	Metric name mismatch, threshold not reached, or evaluation logic not running.	Inspect raw metrics and alert API output; check rule configuration and evaluation cadence.
Missing environment template	The repository may not include .env.example.	Inspect README and configuration code; create .env from documented variables instead of guessing.

8. Maintenance, Security, and Reliability

8.1 Dependency maintenance

```
python -m pip list --outdated
pip freeze > requirements-lock-review.txt
```

Review package updates in a branch, run automated tests, and update the project's dependency file intentionally. Avoid replacing a reviewed requirements file with an unfiltered environment snapshot unless that is the repository's established process.

8.2 Secrets and notification configuration

- Keep `.env` out of Git and provide a safe `.env.example` with variable names but no real credentials.
- Use least-privilege credentials and rotate compromised or exposed webhook URLs and mail credentials immediately.
- Do not hard-code private email addresses, tokens, or webhook URLs in source code, screenshots, or public documentation.
- Prefer a dedicated test channel and test mailbox for demonstrations.

8.3 Logs and incident review

- Use structured logs with timestamps, severity, route template, status code, and a correlation identifier where appropriate.
- Avoid logging passwords, API keys, authorization headers, medical or personal data, and full request bodies by default.
- For production, define log retention and rotation, and forward logs to an approved log platform if needed.

8.4 SLO and histogram review

Review latency percentiles and error rates against an explicit service-level objective. Ensure histogram buckets cover the latency range relevant to the 3-second threshold. A histogram bucket containing observations does not, by itself, prove an alert fired; alert logic must evaluate the relevant metric correctly.

8.5 Production-readiness boundaries

- Use external uptime checks for process or host failure detection.
- Add authentication/authorization to administrative alert and notification-test routes.
- Consider durable alert state and deduplication if alerts must survive restarts or multiple workers.
- Document whether metrics are process-local; multi-worker deployments need an appropriate aggregation strategy.

9. Definition of Done and Handover Checklist

Use this checklist during the final local verification. Check an item only after it has been tested in the current repository version.

■ Application starts from the documented setup steps.

■ Core routes such as /, /health, /users, and /products return their expected responses.

■ The monitoring middleware records request count, duration, and status information.

■ /metrics returns valid metric output.

■ /dashboard loads and updates from the correct API data.

■ /api/alerts reports alert states consistently with configured rules.

■ The 500, slow-response, 401, and 429 scenarios can be reproduced.

■ Slack dispatch has been tested with a non-sensitive test message.

■ Email dispatch has been tested with safe test credentials.

■ RCA notes describe symptoms, evidence, likely cause, and remediation.

■ Secrets are excluded from the repository and public report.

■ README includes setup, architecture, configuration, testing, and known limitations.

10. Deliverables and References

Deliverable	Reference / evidence
Source code	https://github.com/ghkrknkp/FAST-API
Walkthrough recording	https://www.loom.com/share/84959d2b553649eba5a9f97f7b4907fa
Local dashboard	http://localhost:8000/dashboard
Swagger API docs	http://localhost:8000/docs
Metrics endpoint	http://localhost:8000/metrics
Health probe	http://localhost:8000/health

Submission note

Include the repository link and the Loom walkthrough link in the submission message. If the reviewer cannot access localhost, provide screenshots or a recording of the dashboard and test results. Do not describe Slack/email alerts as verified unless you have captured evidence from a successful test run.

Appendix: Quick Start

```
git clone https://github.com/ghkrknkp/FAST-API.git
cd FAST-API
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

After the server starts, open http://localhost:8000/dashboard. If the application layout or entry point differs from the commands above, follow the repository's current README and source structure.